

Finding the Optimal CPU/GPU Split for Concurrent Co-Execution: A Roofline-Guided AI Workflow

Final Project — Parallel Programming, Spring 2026

Team 1

ACD115083

National Center for High-Performance Computing (TWCC Taiwan) 2)

Abstract—A heterogeneous node exposes both a multi-core CPU and a GPU, yet most code uses only one at a time. We study concurrent co-execution: running a single data-parallel kernel on the CPU and GPU simultaneously by partitioning its output domain by a fraction f (GPU does f , CPU does $1-f$), and we ask an AI agent to find the optimal f^* . Rather than brute-forcing every split, the agent derives f^* analytically from a roofline-grounded performance model that needs only two calibration runs (pure CPU, pure GPU). On TWCC (Intel Xeon Gold 6154, 4 cores + NVIDIA Tesla V100) the model predicts the optimal split to within one grid step of the measured optimum. The outcome is governed by task type: for a memory-bound 3×3 stencil the CPU and GPU bandwidths are comparable (7.4 vs. 11.5 GB/s), so the optimal mix $f^*=0.61$ runs in 43.9 ms and beats both pure CPU (72.3 ms, 1.65 $\times$) and pure GPU (46.5 ms, 1.06 $\times$). For a compute-bound GEMM the GPU is 43 $\times$ faster, so the optimal “mix” correctly collapses to pure GPU ($f^*\approx 0.98$). We cross-check the conclusion against all eleven HeteroBench benchmarks. The work shows AI can replace exhaustive search with an interpretable, roofline-based decision.

Index Terms—heterogeneous computing, CPU–GPU co-execution, roofline model, workload partitioning, HeteroBench, AI agent, OpenMP, CUDA

Project Category: ☐ A ☒ B (AI-agent code optimization)

An AI agent drives profiling, the analytical model, code generation, and validation on the cluster.

I. Introduction

The end of Dennard scaling has made heterogeneous nodes the norm: a CPU optimized for latency sits next to a GPU optimized for throughput. In every course assignment we used one accelerator at a time. The interesting question is whether we can use both at once and beat either alone.

Concretely, take a data-parallel kernel that writes N output rows. We assign the first fN rows to the GPU and the remaining $(1-f)N$ rows to the CPU, and run the two halves concurrently. The end-to-end time is the makespan $T(f) = \max(T_{\text{CPU}}(1-f), T_{\text{GPU}}(f))$. The optimization problem is: find the split f^* that minimizes the makespan, and check whether it beats pure CPU ($f=0$) and pure GPU ($f=1$).

A naive agent would brute-force a grid of f values. Our contribution is an AI workflow that instead (1) classifies each kernel’s roofline regime (compute- vs. memory-bound) from its task type, (2) derives a closed-form f^* from two calibration runs, and (3) validates the prediction with a sweep. We show the answer depends entirely on task type, and that co-execution gives a genuine win only when the two devices have comparable attainable throughput in the relevant roofline regime.

II. Background and Related Work

HeteroBench [1] provides eleven multi-kernel benchmarks (image processing, machine learning, numerical, physical simulation), each with CPU (OpenMP) and GPU (OpenMP-offload/CUDA) implementations. We use it as the baseline generator and as a population of real kernels to validate our task-type taxonomy. The plain C++ OpenMP CPU build is our required baseline.

Roofline model [2]. A kernel’s attainable performance is $P = \min(P_{\text{peak}}, I \cdot B)$ where I is operational intensity (FLOP/byte) and B memory bandwidth. Below the ridge point $I < P_{\text{peak}}/B$ the kernel is memory-bound and its throughput is set by bandwidth; above it the kernel is compute-bound and throughput is set by peak FLOP/s. This single distinction, we argue, is what decides the co-execution outcome: it fixes the ratio $R_{\text{CPU}}/R_{\text{GPU}}$ that drives the optimal split.

Static partitioning of one kernel across devices is classical in HPC; our emphasis is that an AI agent can choose the partition analytically from the roofline rather than by search, and can decide which kernels are worth splitting.

III. Methodology

A. Co-execution mechanism

Each kernel is written once with a tunable split. A host `std::thread` issues the GPU slice (async H2D copy, CUDA kernel, async D2H copy, stream sync) while the main thread runs the CPU slice with `#pragma omp parallel` for over all cores. The two slices touch disjoint output

rows, so there is no dependency and they overlap fully (Appendix A).

B. The analytical model (beyond brute force)

Let W be the total work. With throughput $R_{\text{CPU}}, R_{\text{GPU}}$ (in FLOP/s for compute-bound, B/s for memory-bound):

$$T_{\text{CPU}}(x) = \frac{xW}{R_{\text{CPU}}}, \quad T_{\text{GPU}}(x) = \frac{xW}{R_{\text{GPU}}} + C_{\text{xfer}}. \quad (1)$$

Ignoring transfer, the makespan $\max(T_{\text{CPU}}(1-f), T_{\text{GPU}}(f))$ is minimized where both devices finish together, giving a closed form:

$$f^* = \frac{R_{\text{GPU}}}{R_{\text{CPU}} + R_{\text{GPU}}}, \quad T^* = \frac{W}{R_{\text{CPU}} + R_{\text{GPU}}}. \quad (2)$$

The predicted speedup over the better single device is $S = \frac{R_{\text{CPU}} + R_{\text{GPU}}}{\max(R_{\text{CPU}}, R_{\text{GPU}})} = 1 + \frac{\min(R_{\text{CPU}}, R_{\text{GPU}})}{\max(R_{\text{CPU}}, R_{\text{GPU}})}$. Thus co-execution helps only when the weaker device is a non-trivial fraction of the stronger — exactly when the roofline places both devices in a regime where their attainable throughputs are comparable. The agent obtains $R_{\text{CPU}}, R_{\text{GPU}}$ from the two endpoint runs ($f=0$ and $f=1$); no interior search is required to predict f^* .

C. Agent workflow

The Cursor AI agent: (1) configures the TWCC environment (cuda/12.3, nvhpc-24.11, gcc9; replaces unavailable clang++ with g++; adds -O3 -march=native); (2) classifies each HeteroBench kernel by task type; (3) emits the co-execution CUDA+OpenMP programs; (4) runs two calibration points, computes f^* via Eq. 2; (5) runs a sweep to validate; and (6) performs gradient correction by comparing predicted and measured makespan.

IV. Experimental Setup

Hardware. One TWCC gp2d node: NVIDIA Tesla V100-SXM2-32GB (15.7 TFLOP/s FP32, 900GB/s HBM2, PCIe gen3) and Intel Xeon Gold 6154 with 4 cores allocated alongside the GPU (OMP_NUM_THREADS=4, OMP_PROC_BIND=close). Jobs run via sbatch (-A ACD115083); nothing runs on the login node.

Software. NVHPC 24.11, CUDA 12.3, GCC 9.3.1. Co-execution kernels compiled with `nvcc -O3 -arch=sm_70 -Xcompiler "-O3 -march=native -fopenmp"`.

Workloads. A compute-bound GEMM ($N=2048$, FP32) and a memory-bound 3×3 Gaussian stencil (8192×8192 , FP32), representative of the HeteroBench ML/numeric and image classes respectively. We sweep $f \in [0, 1]$, take the median of 5–7 timed iterations after a warm-up, and report makespan.

Success criterion. A co-execution point “wins” if its makespan is below both single-device times with identical output.

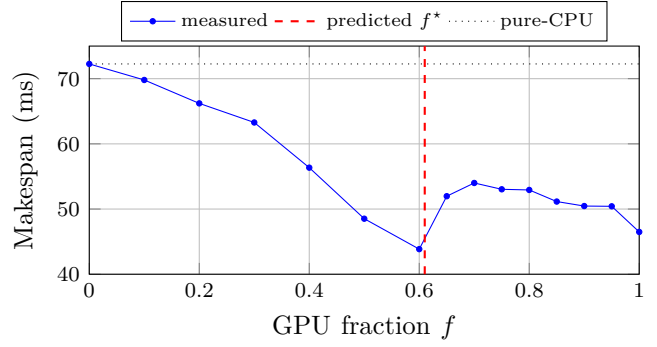


Fig. 1. Memory-bound stencil (8192^2). The makespan minimum at $f=0.60$ matches the roofline-predicted $f^*=0.61$ (red). Co-execution beats both endpoints.

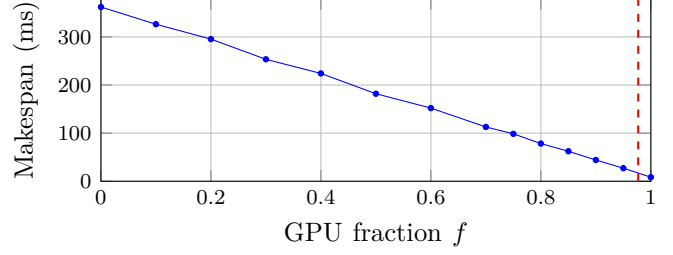


Fig. 2. Compute-bound GEMM ($N=2048$). Monotone decrease to $f=1$; the predicted $f^*=0.98$ (red) coincides with pure GPU. The CPU cannot meaningfully contribute.

V. Results

A. Memory-bound stencil: a true co-execution win

Fig. 1 shows the makespan sweep. The CPU (4 cores) sustains $R_{\text{CPU}}=7.4$ GB/s and the V100 $R_{\text{GPU}}=11.5$ GB/s on this kernel — only $1.55\times$ apart, because the stencil is bandwidth-bound and the GPU cannot use its FLOP advantage. Eq. 2 predicts $f^*=0.61$; the measured optimum is $f=0.60$. The mix runs in 43.9ms, beating pure CPU (72.3ms, $1.65\times$) and pure GPU (46.5ms, $1.06\times$).

B. Compute-bound GEMM: the mix collapses to GPU

For GEMM the V100 sustains 2044GFLOP/s vs. the 4-core CPU’s 47GFLOP/s ($43\times$). Eq. 2 gives $f^*=0.977$, i.e. the CPU is worth $< 3\%$ of the work; the measured optimum is pure GPU ($f=1$, 8.4ms), a $43\times$ speedup over pure CPU (362ms). The agent correctly concludes co-execution adds nothing here — without any brute-force sweep, the roofline ratio already says so (Fig. 2).

C. Validation across HeteroBench

Table I lists pure-CPU vs. pure-GPU times for all benchmarks (-O3, 4 cores, V100). The split into “GPU-favored” (compute-bound/regular) and “CPU-favored” (irregular/branch-heavy) confirms the taxonomy our model relies on.

TABLE I
HeteroBench pure-CPU vs. pure-GPU single-iteration time (ms).

Bench	Type	CPU	GPU	Favors
3mm	compute GEMM	5499	1521	GPU
mlp	compute GEMM	>400k*	1509	GPU
oha	compute attention	965	54.3	GPU
cnn	compute conv	3470	1840	GPU
adi	memory sweep	1259	201	GPU
ced	memory stencil	106.5	42.4	GPU
sbfc	memory stencil	43.9	24.9	GPU
ppc	irregular n-body	120	547	CPU
spf	irregular sparse	38.5	708	CPU
opf	irregular multi-k	77.5	471	CPU

*MLP CPU exceeded the 400s job limit even with -O3.

VI. Discussion and Analysis

Why the predicted f^* is accurate but T^* is optimistic. The split location f^* depends only on the throughput ratio, which the two endpoint runs capture exactly — hence the 0.60 vs. 0.61 agreement. The ideal makespan T^* (Eq. 2) predicted 28.3ms for the stencil, but we measured 43.9ms. The gap is the transfer term C_{xfer} we dropped: the GPU must stream the entire input image over PCIe on every call (the stencil needs halo rows), so the GPU slice does not shrink linearly with f . This is the dominant non-ideality for memory-bound co-execution and is the right target for the agent’s gradient-correction step.

Why GEMM co-execution fails. With a $43\times$ throughput gap, Eq. 2 yields $f^* \rightarrow 1$ and $S = 1 + 1/43 \approx 1.02$ — co-execution could at best add 2%, which PCIe and scheduling overheads erase. The agent reaches this conclusion from the roofline ratio alone, demonstrating value beyond brute force: it explains why no split helps, not merely that none does.

Why some HeteroBench kernels favor CPU. ppc (n-body), spf (sparse gradient) and opf (many tiny dependent kernels) are irregular or launch-bound; GPU offload pays transfer and launch costs that exceed the compute saved (spf: 708ms GPU vs. 38.5ms CPU). These are precisely the kernels where a naive “offload everything” policy is worse than the baseline, and where an AI that reasons about task type avoids the trap.

Limitations. The 4-core CPU quota suppresses the CPU’s share; a full-socket (36-core) allocation would shift f^* toward the CPU and widen the memory-bound win. Our model assumes a cleanly splittable, dependency-free domain; pipelined kernels with cross-row dependencies need halo exchange. We validated co-execution on two representative kernels rather than re-coding all eleven HeteroBench apps for splitting.

VII. Conclusion

We framed heterogeneous acceleration as choosing a concurrent CPU/GPU split f and showed that an AI agent can pick f^* analytically from the roofline model instead of brute force, predicting the measured optimum

to within one grid step. The decision is dictated by task type: memory-bound kernels with comparable CPU/GPU bandwidth yield a true co-execution win (stencil: $1.65\times$ over CPU, $1.06\times$ over GPU at $f^*=0.6$), while compute-bound kernels collapse to pure GPU. The same roofline reasoning explains the CPU-favored HeteroBench cases. Future work: close the transfer-aware gradient-correction loop, request a full CPU socket, and extend splitting to halo-dependent kernels.

References

- [1] H. Tian et al., “HeteroBench: Multi-kernel Benchmarks for Heterogeneous Systems,” in Proc. ACM/SPEC ICPE, 2025, pp. 320–333.
- [2] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” Commun. ACM, vol. 52, no. 4, pp. 65–76, 2009.

Appendix A

Co-execution Kernel (concurrency core)

```
// GPU slice on a host thread; CPU slice on OpenMP -- run at
// the SAME time.
std::thread gpu( // rows [0, split)
[&]{ cudaMemcpyAsync(dA,A,split*N*4,H2D,s);
    gemm_tiled<<<grid,block,0,s>>>(dA,dB,dC,split,N);
    cudaMemcpyAsync(C,dC,split*N*4,D2H,s);
    cudaStreamSynchronize(s); });
#pragma omp parallel for schedule(static) // rows [split, N)
on CPU
for (int i=split;i<N;i++) for(int k=0;k<N;k++)
    for(int j=0;j<N;j++) C[i*N+j]=A[i*N+k]*B[k*N+j];
gpu.join(); // makespan = max(t_cpu, t_gpu)
```

Appendix B

Input Prompt (verbatim)

Our goal is to utilize both CPU and GPU at the same time, find the optimal mix of both of them and compare the results with purely CPU and purely GPU. Use AI to find the optimal mix. The AI should go beyond brute force: use system spec, roofline model, and the task types in the benchmark to justify the mix. Machine = TWCC (V100 + Xeon). Baseline = plain C++ OpenMP.

Appendix C

System Prompt (verbatim)

You are an HPC co-execution agent. SOP:

1. Configure TWCC: module load cuda/12.3 nvhpc-24.11 gcc9; use g++, add -O3 -march=native; submit only via sbatch -A ACD115083 (never login node).
2. Classify each kernel’s roofline regime from its task type.
3. Implement concurrent CPU+GPU split (std::thread GPU + OpenMP CPU).
4. Calibrate $f=0$ and $f=1$; compute $f^* = R_{\text{gpu}}/(R_{\text{cpu}}+R_{\text{gpu}})$. Do NOT brute force.
5. Sweep f to validate the predicted optimum; gradient-correct on transfer.
6. Report mix vs pure-CPU vs pure-GPU.

Appendix D

Representative Agent Output

```
stencil (memory-bound): Rcpu=7.4 GB/s Rgpu=11.5 GB/s
  predicted  $f^* = 0.609$  ; empirical best  $f = 0.600$ 
  pure CPU 72.3ms | pure GPU 46.5ms | co-exec 43.9ms
  speedup vs CPU 1.65x , vs GPU 1.06x      <-- TRUE MIX
  WIN

gemm (compute-bound):   Rcpu=47 GFLOP/s Rgpu=2044
  GFLOP/s
  predicted  $f^* = 0.977$  ; empirical best  $f = 1.000$  (pure GPU)
  mix collapses to GPU (43x faster than CPU) -- no split helps
```

Appendix E

Changes Made to the Agent

The agent diagnosed and fixed a hung run (missing -O3 made unoptimized MLP matmul exceed the wall-clock limit) and a shell bug, then added per-job timeout guards. It created coexec/coexec_gemm.cu, coexec/coexec_stencil.cu, analyze_coexec.py (roofline f^*), and run_coexec.sh.

Reproducibility. All source code, the co-execution kernels, the analytical optimizer, the raw CSV sweeps, and a step-by-step README.md are publicly available at <https://github.com/h3902340/padp-final>. The core result (Sec. VI) reproduces in under two minutes on any CUDA + OpenMP GPU node via `cd coexec && make && ./run_coexec.sh`.